

ABIDE ASD DETECTION PIPELINE

Updated Comprehensive Academic & Technical Project Report

AI-Based Autism Spectrum Disorder Classification from Resting-State fMRI
Functional Connectivity

Updated deployment contract

The interactive application is specified to accept exactly three input formats: .nii, .jpg, and .png. Other file types are rejected. The core production model and benchmark are kept unchanged in the revised deployment design.

This report consolidates the supplied project documentation and incorporates the updated input-format requirement into the software architecture, inference workflow, validation checks, and deployment discussion. The production benchmark remains the previously reported RBF-SVM result.

Project Component	Implementation
Dataset	ABIDE-I / ABIDE-II framework
Brain parcellation	Craddock 200 (CC200), 200 ROIs
Connectivity	Pearson correlation + Fisher arctanh
Initial feature space	19,900 unique connectivity edges
Feature selection	Welch two-sample t-test; top 600 edges
Production classifier	RBF Support Vector Machine, C = 10.0
Deployment	Streamlit dashboard; .nii / .jpg / .png input contract

Reported production benchmark: Accuracy 94.8315%, Balanced Accuracy 94.8468%, ROC-AUC 0.9319, Sensitivity 95.3704%, Specificity 94.3231%, F1-Score 0.9471, MCC 0.8967.

Abstract and Key Performance Summary

Autism Spectrum Disorder (ASD) is a heterogeneous neurodevelopmental condition, and the project described in this report investigates whether resting-state functional magnetic resonance imaging (rs-fMRI) can provide a useful quantitative representation for binary classification. The central design converts regional brain time-series into a functional-connectivity graph, reduces the resulting high-dimensional edge space through statistical ranking, and uses a nonlinear Support Vector Machine (SVM) as the production classifier.

The production pipeline uses the Craddock 200 (CC200) atlas to represent 200 functional regions of interest. Regional signals undergo 5th-95th percentile clipping and standardization. Pearson correlation produces a 200 x 200 connectivity matrix, Fisher arctanh transformation stabilizes the edge representation, and the 19,900 unique upper-triangular connections are ranked using Welch two-sample t-tests. The top 600 edges are standardized and supplied to an RBF-kernel SVM with $C = 10.0$ and balanced class weights.

The supplied benchmark records 94.8315% accuracy, 94.8468% balanced accuracy, 0.9319 ROC-AUC, 95.3704% sensitivity, 94.3231% specificity, 0.9471 F1-score, and 0.8967 MCC. These figures describe the stated evaluation configuration and are not evidence of clinical diagnostic validity or universal generalization.

Metric	Achieved value
Accuracy	0.9483146067415731 (94.8315%)
Balanced Accuracy	0.9484675723758693 (94.8468%)
ROC-AUC	0.9319100760148795
Sensitivity	0.9537037037037037 (95.3704%)
Specificity	0.9432314410480349 (94.3231%)
F1-Score	0.9471264367816092
MCC	0.8966632720696869

Document Scope, Revision Note, and Intended Use

This revised edition is based on the supplied 49-page technical report and retains its terminology, equations, model inventory, performance values, software modules, and deployment concepts. The main change is the explicit treatment of the updated application input contract: the dashboard is required to accept .nii, .jpg, and .png files, and no other formats. The revised report therefore adds a dedicated input-adapter layer and a set of validation and regression requirements around that change.

Accuracy preservation requirement

The input-format extension is an interface and preprocessing concern. The production RBF-SVM, its selected feature mapping, its StandardScaler parameters, and the benchmark configuration are treated as protected components. Any code modification needed to admit the new formats should be isolated from the learned classifier so that the reported benchmark is not silently changed.

The project remains a research prototype. A prediction is a model output, not a clinical diagnosis. The report intentionally avoids describing the classifier as a medical device or claiming that the benchmark transfers to new hospitals, scanners, demographics, or clinical populations without independent validation.

The report also distinguishes measured results from research architectures. DNFN, HGSO, FHGO-DNFN, FHJO-CNN, EfficientNet-B0, SAE, and SAE-FRNN are documented as experimental branches. The 94.8315% production benchmark belongs to the evaluated RBF-SVM pipeline described here.

Table of Contents

Section	Pages
Front Matter	1-6
1. Introduction and Clinical Background	7-11
2. Dataset and Neuroimaging Representation	12-15
3. Preprocessing and Feature Engineering	16-24
4. Production RBF-SVM	25-33
5. Advanced Research Architectures	34-43
6. Software Architecture and Deployment	44-52
7. Ethics, Limitations, and Future Work	53-54
8. Conclusion and References	55

Detailed contents: Abstract and Key Performance Summary; Document Scope; Abbreviations; Executive Summary; Clinical Background; rs-fMRI; Motivation; Objectives; Dataset; Data Organization; CC200; ROI Time-Series; Signal Preprocessing; Percentile Clipping; Standardization; Functional Connectivity; Fisher Transformation; Edge Vector; Pivotal Feature Selection; Leakage and Validation; Feature Scaling; SVM Architecture; Mathematical Formulation; Hyperparameters; Training Protocol; Metrics; Results; Confusion Matrix; ROC-AUC; Error Analysis; DNFN; Fuzzy Rules; HGSO; FHGO-DNFN; JSO; FHJO-CNN; EfficientNet-B0; Sparse Autoencoder; FRNN and GRU; Model Comparison; Software Modules; Directory; Dashboard; Updated Input Contract; Input Adapters; Outputs and Validation; Accuracy Preservation; Error Handling; Execution; Ethics; Limitations; Future Work; Conclusion and References.

Abbreviations and Notation

Term	Meaning
ABIDE	Autism Brain Imaging Data Exchange
ASD	Autism Spectrum Disorder
CON	Typical Control
rs-fMRI	Resting-State Functional Magnetic Resonance Imaging
BOLD	Blood-Oxygen-Level-Dependent
ROI	Region of Interest
CC200	Craddock 200 functional parcellation atlas
FC	Functional Connectivity
SVM	Support Vector Machine
RBF	Radial Basis Function
DNFN	Deep Neuro-Fuzzy Network
HGSO	Henry Gas Solubility Optimization
JSO	Jellyfish Search Optimization
GRU	Gated Recurrent Unit
SAE	Sparse Autoencoder

Notation used throughout the mathematical sections includes T for the number of temporal observations, N for the number of ROIs, M for the number of unique connectivity edges, K for the selected number of pivotal edges, and n for the number of subjects. The symbol ϵ denotes a small numerical constant used to avoid unstable division or logarithms.

Executive Summary

The ABIDE ASD Detection Pipeline brings together neuroimaging preprocessing, statistical feature engineering, nonlinear classification, advanced research models, and a lightweight deployment layer. Its practical center is not a single neural network, but a carefully staged transformation from regional fMRI time-series to a compact set of 600 discriminative connectivity features.

The first engineering decision is to work at the ROI level. A 200-region CC200 representation is far easier to reason about than raw voxel data, while still retaining network-level information. Each subject is converted to 19,900 unique pairwise connections, creating a $p \gg n$ setting that would be difficult for a conventional classifier without aggressive reduction.

The second decision is statistical ranking. Welch t-tests identify the edges with the largest group-level separation, after which only the top 600 are retained. This gives the production classifier a stable, compact input space and also creates a natural bridge to the experimental neuro-fuzzy and attention-based branches.

The third decision is deployment discipline. The serialized production pipeline is treated as the source of truth for inference. The updated Streamlit interface adds support for exactly .nii, .jpg, and .png inputs through an input-adaptor layer while preserving the trained SVM behavior. Regression checks therefore focus on both file acceptance and benchmark preservation.

1. Introduction and Clinical Background

Autism Spectrum Disorder is characterized by differences in social communication together with restricted or repetitive patterns of behavior. The condition is highly heterogeneous, which makes it unlikely that a single biological measurement will fully represent every individual. Neuroimaging research therefore focuses on patterns of brain organization that may differ between groups rather than searching for a single deterministic marker.

Resting-state fMRI is particularly useful for computational studies because it records spontaneous fluctuations while the participant is not required to perform a particular task. The project uses those temporal fluctuations to estimate statistical relationships between brain regions. Functional connectivity then becomes the bridge between the imaging signal and machine-learning representation.

The intended contribution of the system is technical: it demonstrates an end-to-end workflow that is reproducible enough to inspect, extend, and deploy as a research prototype. The report treats prediction performance, feature selection, and architecture choices as engineering objects that must be understood together rather than as isolated steps.

The same discipline is applied to the updated interface. Expanding the accepted file formats is not treated as permission to rewrite the classifier. Instead, file handling is separated from learned parameters, allowing the interface to evolve while the production decision function remains protected.

From a systems perspective, the clinical background explains why the engineering target is not a single image label. The project works with distributed relationships, so the chosen representation has to preserve temporal information long enough to estimate interactions between regions. This choice also explains why the report gives equal attention to preprocessing and classification. In neuroimaging, a model can be technically correct while operating on a representation that was assembled inconsistently; the resulting number would still be misleading. The pipeline therefore treats each transformation as part of the experiment rather than as routine data preparation.

2. Autism Spectrum Disorder and Neuroimaging

Neuroimaging-based ASD research often studies distributed networks because many cognitive functions depend on interactions among multiple brain systems. In a functional-connectivity view, each ROI is a node and each pairwise relationship is an edge. The resulting graph can be analyzed statistically, visually, or through a classifier.

This project focuses on large-scale connectivity. It does not claim that a selected edge is a causal mechanism of ASD. A statistically discriminative edge simply shows stronger separation between the two groups under the chosen representation and sampling conditions. That distinction is important when a computational result is translated back into neuroscience.

The pipeline also recognizes practical sources of variation. Scanner hardware, acquisition sequences, participant motion, demographics, site effects, and preprocessing decisions can all alter the observed signal. A high score on one split can therefore be useful as a project benchmark without being sufficient evidence for clinical deployment.

Interpretation principle

The model identifies patterns that help classification. It does not, by itself, establish causation, diagnostic specificity, or biological exclusivity.

A useful way to read the project is as a network-analysis problem expressed through machine learning. The classifier does not observe a named clinical symptom directly. It observes numerical summaries of interactions among atlas-defined regions. This makes interpretation multi-level: an individual edge can be inspected, several edges can be grouped into a network, and the complete selected set can be evaluated for stability. The report intentionally keeps the language cautious because the same connectivity pattern can be influenced by acquisition conditions, subject composition, and preprocessing choices.

3. Resting-State fMRI as the Input Modality

A resting-state fMRI scan records a sequence of brain volumes across time. After a compatible preprocessing and parcellation process, each CC200 region can be represented by a single temporal signal. The subject is then represented as a matrix X in $\mathbb{R}^{(T \times 200)}$, where T is the number of time points.

The project does not feed raw voxels directly into the production SVM. Instead, the scan is converted to an ROI representation, cleaned, standardized, and transformed into connectivity features. This distinction becomes important for the deployment extension because a .nii volume and a .jpg image are not intrinsically the same mathematical object as a 200-ROI time-series.

The revised software design therefore introduces an explicit input-adapter concept. The dashboard may accept the three allowed file suffixes, but the adapter is responsible for validating the content and converting it into the representation required by the downstream model path. The classifier itself remains unchanged.

The distinction between image data and time-series data becomes especially important in deployment. A NIfTI object can represent a three-dimensional volume or a four-dimensional time series, whereas JPEG and PNG are ordinary image containers. The extension requirement is therefore a software contract, not a scientific claim that all three files contain equivalent information. The adapter layer has to decide how a valid file is represented before the protected classifier can be used. This keeps the interface flexible without quietly changing the meaning of the model input.

4. Motivation for a High-Dimensional Connectivity Pipeline

With $N = 200$ ROIs, there are $N(N-1)/2 = 19,900$ unique undirected connections. This is already large compared with the number of subjects in a typical ABIDE subset. Feeding all edges into a classifier can encourage overfitting, increase training cost, and make interpretation harder.

The project responds by combining statistical reduction with a nonlinear classifier. The statistical stage asks a simple but useful question: which individual edges show the largest separation between ASD and control groups? The classifier then operates only on those selected edges, where the feature dimension is 600 rather than 19,900.

This reduction is also a design choice for deployment. A saved model does not need to recompute every possible decision about the feature space at run time; it needs the same edge mapping and scaling parameters that were used during training. Preserving that mapping is therefore as important as preserving the SVM itself.

The dimensionality argument also explains the emphasis on statistical selection. With 19,900 candidate edges, many measurements will be correlated because neighboring or shared ROIs participate in several connections. A classifier can exploit such redundancy, but without careful reduction the model may memorize sample-specific structure. Selecting a compact set of high-separation edges is therefore both a computational and interpretive decision. The 600-edge representation is a practical compromise within the supplied experiment.

5. Research Objectives and System Requirements

- Build a modular rs-fMRI processing pipeline using the CC200 atlas.
- Convert regional signals into Fisher-transformed functional-connectivity features.
- Rank and retain the top 600 group-discriminative edges.
- Train and evaluate an RBF-SVM using balanced class weighting.
- Retain exact model artifacts required for repeatable inference.
- Provide evaluation outputs including accuracy, balanced accuracy, ROC-AUC, sensitivity, specificity, F1, MCC, and a confusion matrix.
- Document DNFN, HGSO, FHGO-DNFN, FHJO-CNN, EfficientNet-B0, SAE, and SAE-FRNN research branches.
- Update the dashboard input contract so that only .nii, .jpg, and .png files are accepted.
- Keep the production classifier and previously reported benchmark protected during the interface update.
- Ensure invalid input types and malformed files fail cleanly rather than generating undefined model behavior.

The resulting system is best viewed as a research platform with a production-oriented baseline. The software architecture is intentionally modular so that a file-format adapter can evolve without rewriting the core feature extraction or classifier implementation.

The objectives are deliberately split between scientific processing and software behavior. The scientific side concerns how brain signals become features and how those features support classification. The software side concerns repeatability, saved artifacts, input validation, and stable inference. The updated file-format requirement sits on the software side, which is why the report treats it as an adapter and validation problem rather than as a reason to change the learning algorithm.

6. Dataset and ABIDE Research Context

The Autism Brain Imaging Data Exchange (ABIDE) provides a multisite research resource containing resting-state fMRI and associated phenotypic information for ASD and typical-control participants. The project is structured to accommodate ABIDE-I and ABIDE-II, while each reported experiment must still identify the exact subset, file preparation, and inclusion criteria used.

Multisite data improves diversity, but it also creates opportunities for unwanted site-specific cues. Differences in scanners, acquisition parameters, participant composition, and preprocessing can become detectable by a machine-learning model. This is why the report treats external validation and site-held-out testing as future requirements rather than optional extras.

The GitHub repository should contain code, configuration, documentation, and non-sensitive artifacts rather than raw subject data. The actual subject files remain outside version control when dataset terms or privacy considerations require that separation.

Reproducibility record

A trustworthy experiment should record the dataset version, inclusion rules, preprocessing source, atlas, split, random state, selected-edge count, SVM hyperparameters, package versions, and model artifact hash.

The multisite nature of ABIDE is valuable because it broadens the subject population, but it also means that a model may encounter distributions it did not see during training. Site differences can enter through scanner physics, acquisition protocols, preprocessing, recruitment patterns, and demographics. A benchmark on a random split is therefore useful for measuring the current implementation, while site-held-out testing is more informative for judging generalization. The report keeps these two claims separate.

7. Data Organization and Subject Matching

The data-ingestion layer is responsible for discovering functional files and associating them with subject-level metadata. Because ABIDE data may be organized into site and subject subdirectories, recursive discovery is used rather than assuming a single fixed folder depth.

```
data/  
  raw/  
    abide1_data.csv  
    abide2_data.csv  
  ABIDE/  
    site_1/  
    site_2/  
  processed/  
  features/
```

Subject matching is an important point of failure. A file that cannot be linked to a known subject label cannot safely enter supervised training. The loader should therefore validate identifiers and fail explicitly when the input folder is empty, the file pattern is wrong, or the metadata mapping is incomplete.

The same validation philosophy is used in deployment. A user may upload a file with the correct extension but invalid content. The dashboard should distinguish extension validation from content validation and from downstream model compatibility.

Subject matching should be viewed as part of the supervised learning problem rather than simple file housekeeping. A wrong identifier can pair a scan with the wrong label, creating a clean-looking but incorrect training example. Recursive discovery reduces assumptions about folder layout, while explicit validation reduces silent failures. The same principle applies to the dashboard: a readable upload is not enough; the system must confirm that the contents can be transformed into a model-compatible object.

8. CC200 Atlas and ROI Representation

The Craddock 200 atlas provides a functional parcellation of the brain into 200 regions. The atlas defines the coordinate system of the connectivity problem: every ROI is a node, every pair of ROIs defines a possible edge, and every selected feature can ultimately be traced to an ROI pair.

For a subject, the regional time-series matrix is represented as X in $\mathbb{R}^{(T \times 200)}$. Column j contains the temporal trace for ROI j . This is substantially more compact than raw voxel data and creates a representation that can be compared across subjects after suitable preprocessing.

The atlas is therefore part of the model definition. Replacing CC200 with a different parcellation changes the feature identity, the number of edges, and the meaning of any selected feature. The saved 600-edge mapping is only valid for the atlas and edge-order convention used during training.

Because every feature ultimately refers to a pair of CC200 regions, the atlas establishes a stable vocabulary for the entire pipeline. The choice of atlas affects the number of nodes, the number of edges, and the biological interpretation of selected features. Any change in parcellation therefore requires a full regeneration of the connectivity matrix, edge ordering, feature ranking, and model. The atlas should be versioned alongside the model artifact for reproducibility.

9. ROI Time-Series and Matrix Construction

The starting point for the production feature pipeline is a valid ROI matrix. The expected mathematical object is a two-dimensional array with a temporal axis and a 200-region axis. Data quality checks should verify that the array is numeric, finite, and large enough to produce meaningful correlation estimates.

Object	Shape / property
Subject ROI matrix	T x 200
Single ROI signal	T
Connectivity matrix	200 x 200
Unique undirected edges	19,900
Selected pivotal edges	600

Keeping the shape contract explicit prevents subtle bugs. A row-major versus column-major interpretation, an unexpected ROI count, or an accidentally flattened image can all produce numerically valid arrays that are nevertheless semantically wrong. The implementation should validate shape before feature extraction.

Shape validation is particularly important in neuroimaging pipelines because many arrays are numerically compatible with several interpretations. For example, an array can be transposed without producing an immediate error, yet the temporal and ROI axes would be reversed. The implementation should therefore check expected dimensions, finite values, and the number of ROIs before calculating correlations. These checks are cheap compared with the cost of training on incorrectly shaped data.

10. Signal Preprocessing Overview

The documented preprocessing path uses percentile clipping followed by ROI-wise standardization. The intent is not to erase the temporal structure, but to reduce the influence of extreme signal values before estimating pairwise relationships.

For each ROI j , the lower and upper clipping thresholds are the 5th and 95th percentiles. The temporal index is preserved. After clipping, each ROI is centered and scaled using its own mean and standard deviation.

This stage sits before functional connectivity. Any preprocessing that occurs after the model has learned its feature mapping would change the representation seen by the SVM. For that reason, the exact order of operations is part of the reproducibility specification.

The order of preprocessing operations is not arbitrary. Clipping changes extreme values, standardization rescales each regional trace, and correlation is then estimated from those standardized signals. Moving one operation earlier or later can change the resulting edge values. For this reason, a production model should preserve the preprocessing recipe in code and documentation, and any adapter introduced for a new file format should reuse the same downstream sequence wherever possible.

11. Percentile-Based Outlier Clipping

The clipping transformation is defined as:

$$x_clean(t,j) = \min(\max(x(t,j), P_{0.05}(x_j)), P_{0.95}(x_j))$$

Values below the fifth percentile are lifted to the lower threshold and values above the ninety-fifth percentile are reduced to the upper threshold. The method therefore changes magnitudes rather than deleting time points.

The choice of percentile limits is part of the project specification. It should not be changed casually after the benchmark has been established because any change in the regional traces can propagate through correlation, Fisher transformation, feature ranking, and ultimately the SVM.

Implementation rule

The updated file-format support must not silently introduce a different clipping or normalization policy. Input adapters should feed the same downstream preprocessing logic whenever the target representation is the production FC pipeline.

Percentile clipping is attractive here because it preserves the number and position of temporal observations. This matters for correlation: deleting time points separately from different ROI signals could create misalignment and distort the estimated relationship between regions. Clipping instead limits the influence of unusually large or small observations while keeping the temporal grid intact. The chosen 5th and 95th percentiles are therefore both a signal-processing parameter and part of the reproducibility record.

12. ROI-Wise Standardization

After clipping, each ROI signal is standardized independently:

$$\hat{x}(t,j) = [x_{\text{clean}}(t,j) - \mu_j] / [\sigma_j + \epsilon]$$

The small constant $\epsilon = 10^{-6}$ prevents unstable division when a signal has very small variance.

Standardization makes the subsequent correlation computation insensitive to differences in signal scale between ROIs.

This local normalization step is different from the later StandardScaler used on the 600 selected connectivity features. The first operates on temporal ROI signals within a subject; the second operates across subjects on the final model features. Keeping those two operations conceptually separate helps avoid implementation mistakes.

Two standardization layers are present in the overall workflow and they serve different purposes. ROI-wise standardization makes regional time-series comparable before correlation. The later StandardScaler normalizes the selected edge features across the training cohort before classification. Confusing these two operations can create subtle discrepancies between training and inference. The report therefore treats both as explicit contracts, with the learned feature scaler stored for reuse.

13. Functional Connectivity Computation

Functional connectivity is estimated using Pearson correlation between every pair of standardized ROI signals. Because the signals have already been centered and scaled, the coefficient can be expressed as a normalized dot product:

$$r_{ij} = \sum_t \hat{x}(t,i) \hat{x}(t,j) / T$$

The complete result is a symmetric 200 x 200 matrix R . The diagonal contains self-correlation and is not used for prediction. Since $r_{ij} = r_{ji}$, only one triangular half contains unique information.

From a graph perspective, the 200 ROIs are nodes and the correlation values are weighted edges. This graph interpretation is helpful when visualizing selected features and when connecting the production pipeline to the later attention and neuro-fuzzy research branches.

The functional-connectivity matrix can be viewed as a weighted graph. This viewpoint is useful because it explains why the upper triangle is sufficient and why each edge can later be mapped back to an ROI pair. It also creates a natural bridge to the research models: the SVM consumes a vectorized graph, while the attention-based branch can preserve a two-dimensional arrangement and the neuro-fuzzy branch can operate on the selected numerical representation.

14. Fisher arctanh Transformation

Correlation values are bounded between -1 and 1 and have a sampling variance that depends on the magnitude of the correlation. The project applies Fisher transformation to obtain a more statistically convenient representation for group-level comparisons.

$$z_{ij} = \text{arctanh}(\text{clip}(r_{ij}, -0.9999, 0.9999))$$

Equivalently: $z = 1/2 \ln((1+r)/(1-r))$. The strict clipping step prevents numerical overflow as r approaches either boundary. After transformation, the matrix remains symmetric, and the upper triangle still contains 19,900 unique edge values.

Fisher transformation does not remove features. It changes their numerical scale before the t-test stage. The actual reduction from 19,900 to 600 happens only after the group-level statistical ranking.

Fisher transformation is a numerical representation step, not a feature-selection algorithm. The clipping of correlations before arctanh is a practical safeguard against values that approach the mathematical singularity at absolute correlation one. Once transformed, the edges can be compared across subjects with a more convenient scale. The later t-test is what determines which connections become part of the 600-feature production representation.

15. Construction of the 19,900-Edge Feature Vector

For $N = 200$, the number of unique pairwise connections is:

$$M = N(N-1)/2 = 200 \times 199 / 2 = 19,900$$

For each subject, the upper triangle of the Fisher-transformed connectivity matrix is flattened into a vector V in $\mathbb{R}^{19,900}$. The order of that flattening is a hidden but critical contract because the selected indices refer to positions in this vector.

A production artifact must therefore preserve not just the SVM weights but also the exact edge-index list. Without the edge mapping, the model may receive the correct number of features but attach each number to the wrong ROI pair, producing predictions that are technically valid and scientifically meaningless.

The edge ordering deserves special attention in deployment. A model trained on one ordering cannot be safely used with another ordering even when both contain exactly 19,900 numbers. The selected index list is therefore part of the learned artifact. A robust implementation should treat the pair-to-index mapping as immutable metadata, not as a calculation that can be recreated independently with a potentially different loop order.

16. Global Pivotal Feature Selection

The high-dimensional vector is reduced using an independent Welch two-sample t-test for every edge. The test compares the ASD and control distributions without assuming equal variances.

$$t_k = (\text{mean_ASD} - \text{mean_CON}) / \sqrt{(\text{var_ASD}/n_{\text{ASD}} + \text{var_CON}/n_{\text{CON}})}$$

Edges are ranked using the absolute statistic $|t_k|$. A large absolute value indicates a stronger difference between the group means under the chosen sampling configuration, regardless of the direction of the effect.

The top $K = 600$ edges are retained. This is the core dimensionality-reduction step and also the main statistical interpretation layer of the production branch.

Welch testing is used because it allows the two class distributions to have different variances. The project ranks by absolute t-statistic, so the direction of the group difference is not used during selection. Direction can still be retained as metadata for later interpretation. In a research setting, that distinction is useful: ranking answers how strongly a feature separates groups, while direction answers which group has the larger mean transformed connectivity value.

17. Pivotal-Edge Ranking and Selection Logic

The selection operation can be written as:

$$S^* = \operatorname{argmax}_{|S|=600} \sum_{k \in S} |t_k|$$

Operationally, the implementation can simply sort the absolute t-statistics and take the first 600 indices. The optimization notation expresses the same selection rule in a compact mathematical form.

Stage	Input	Output
Connectivity	200 x 200 matrix	19,900 unique edges
Statistical testing	ASD/CON edge values	19,900 t-statistics
Ranking	t values	Sorted edge indices
Selection	Sorted list	Top 600 pivotal edges

The selected features are not automatically biomarkers. They are candidate discriminative connections that must be checked for stability, site effects, and biological plausibility before stronger conclusions are drawn.

The top-600 choice is a dimensionality-control decision. Reducing by a fixed count rather than an arbitrary p-value threshold produces a predictable model input size and a simple deployment contract. It also makes comparisons easier because every subject enters the production classifier through exactly 600 positions. If the selection count is changed later, the model must be retrained because both the feature mapping and the decision boundary depend on the selected set.

18. Leakage, Feature Selection, and Validation Discipline

High-dimensional feature selection introduces a specific validation risk. If all labels are used to rank the 19,900 edges before a train/test split, information from the evaluation set can influence which features are selected. The model is then evaluated on a feature representation that already contains indirect knowledge of the test labels.

Leakage-safe protocol

For a future rigorous benchmark, t-test ranking should be learned within the training portion of each fold. The held-out fold should only be transformed using the edge indices obtained from the corresponding training data.

The same principle applies to scaling: StandardScaler parameters must be fitted on training data only. These rules are especially important if the model is compared across repeated splits or sites.

The reported 94.8315% result should therefore be interpreted as the benchmark of the supplied experiment, while leakage-safe nested validation is a methodological improvement recommended for future evaluation.

The leakage discussion is important because feature selection uses labels. If the entire dataset is used to identify the strongest edges before the split, the evaluation set is no longer fully unseen. The reported score can still be recorded as the supplied experiment, but a stronger future design should embed feature ranking inside the training folds. That change would improve methodological validity without changing the scientific goal of reducing 19,900 features to a compact representation.

19. Feature Scaling and Final Model Input

Once the 600 pivotal edges have been selected, the final feature matrix has the shape $n \times 600$. A `StandardScaler` is fitted on the training cohort so that each selected feature is approximately centered and normalized before SVM fitting.

The scaler parameters form part of the deployment artifact. At inference time, the same means and scales must be reused; they must not be recomputed from a single new subject or from the full evaluation set.

Representation	Dimension
ROI signals	$T \times 200$
Full FC vector	19,900
Production input	600 standardized features

This compact 600-feature representation is the protected interface of the production classifier. Any file-format adapter added to the application must ultimately deliver a numerically compatible vector to this stage if the SVM is to remain unchanged.

In deployment, the `StandardScaler` is effectively part of the model. The SVM expects the same numerical scale it saw during training, so refitting the scaler on a new subject would alter the coordinate system of the decision function. The saved edge indices and saved scaler together form the bridge between training and inference. This is also why input adapters must terminate at a representation compatible with the existing 600-feature contract.

20. Production RBF-SVM Architecture

The production classifier is a Support Vector Machine with a Radial Basis Function kernel. The model is trained on the selected and standardized 600-edge representation. An RBF kernel is appropriate for the project because the relationship between connectivity features and the binary target is not assumed to be linearly separable.

$$K(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2)$$

The documented production configuration uses $C = 10.0$ and balanced class weights. The model therefore combines nonlinear decision geometry with class-sensitive regularization.

Compared with the deep-learning branches, the SVM is relatively lightweight at inference time. That practical characteristic is one reason it serves as the production baseline while the more elaborate architectures remain available for research experiments.

The RBF kernel measures similarity through squared Euclidean distance in the standardized feature space. Rather than fitting a straight separating hyperplane, the SVM can form a nonlinear boundary that bends around the observed samples. This choice is practical for a functional-connectivity problem because discriminative information may be distributed across edges in ways that are not linearly separable after statistical selection.

21. SVM Mathematical Formulation

The soft-margin SVM can be expressed in dual form as the optimization:

maximize $\sum_i \alpha_i - \frac{1}{2} \sum_i \sum_j \alpha_i \alpha_j y_i y_j K(\mathbf{x}_i, \mathbf{x}_j)$

subject to $0 \leq \alpha_i \leq C_i$ and $\sum_i \alpha_i y_i = 0$.

The support vectors are the training observations with non-zero multipliers. They determine the learned decision boundary. The RBF kernel implicitly maps the original 600-dimensional vector into a richer feature space without requiring the transformed coordinates to be explicitly constructed.

Model boundary

The SVM is the component responsible for the reported production benchmark. The research models described later should not be substituted for this result unless they are separately trained and evaluated.

The dual formulation clarifies why only selected observations become influential support vectors. The learned boundary is controlled by the examples with non-zero Lagrange multipliers, while the RBF kernel determines how those examples interact in the implicit feature space. This provides a mathematically compact way to represent a nonlinear classifier without constructing a new feature dimension for every kernel coordinate.

22. Hyperparameters and Class Weighting

Parameter	Documented configuration
Classifier	Support Vector Machine
Kernel	RBF
Regularization C	10.0
Class weighting	Balanced
Input dimension	600 selected features
Kernel scale	Variance-scaled expression based on 600-feature input

Balanced weighting assigns different effective penalties based on class frequency. The conceptual form documented for the project is $C_i = C * n / (2 n_{yi})$, where n_{yi} is the number of training observations in class y_i .

These values are part of the benchmark definition. A seemingly harmless hyperparameter change can move the decision boundary and invalidate direct comparison with the reported result. That is why the updated file-input work is explicitly required to preserve them.

Hyperparameters should be recorded as part of the benchmark, not merely as code defaults. The value $C = 10.0$ controls how the optimization trades margin width against training errors, while balanced weighting changes the effective penalty assigned to each class. Even when the dataset is not severely imbalanced, explicit class weighting makes the decision policy clear. Preserving these settings is a direct part of the accuracy-maintenance requirement.

23. Training and Evaluation Protocol

The reported experiment uses an 80/20 stratified split. Stratification keeps the proportion of ASD and control subjects similar in the training and held-out portions.

The conceptual production sequence is: load data, preprocess signals, compute functional connectivity, apply Fisher transformation, construct edge vectors, select pivotal edges, standardize selected features, fit the RBF-SVM, evaluate, and serialize the model artifacts.

The revised deployment architecture does not retrain the model merely because the dashboard accepts new file types. The application layer and its adapters are expected to change independently. This is central to the instruction that the earlier accuracy remain maintained.

A complete reproducibility record should include the exact split, random seed when applicable, dataset subset, preprocessing source, atlas, feature count, hyperparameters, environment versions, and artifact checksum.

A stratified split provides a reproducible snapshot of model behavior but not a complete generalization study. The supplied benchmark is useful as a baseline for the repository, particularly because the evaluation metrics have been recorded to many decimal places. For a publication-quality claim, the same pipeline should also be tested under repeated splits and independent sites. The important point is that the implementation and the evaluation protocol must be treated as one unit.

24. Evaluation Metrics and Their Meaning

Metric	Formula / interpretation	Value
Accuracy	$(TP+TN)/(TP+TN+FP+FN)$	94.8315%
Balanced Accuracy	Average of sensitivity and specificity	94.8468%
Sensitivity	$TP/(TP+FN)$	95.3704%
Specificity	$TN/(TN+FP)$	94.3231%
ROC-AUC	Area under ROC curve	0.9319
F1-Score	$2PR/(P+R)$	0.9471
MCC	Correlation-like binary metric	0.8967

The use of multiple metrics avoids relying on one aggregate number. Sensitivity focuses on ASD detection, specificity focuses on control recognition, balanced accuracy reduces the influence of class imbalance, ROC-AUC summarizes ranking discrimination, and MCC provides a compact measure of agreement that remains informative when class sizes differ.

The selected metric set covers complementary failure modes. Accuracy gives an overall proportion of correct predictions; balanced accuracy checks that performance is not driven by class distribution; sensitivity focuses on ASD detection; specificity focuses on control recognition; F1 summarizes precision and recall; ROC-AUC evaluates ranking over thresholds; and MCC provides a correlation-style summary of binary agreement. Reporting all seven reduces the chance that one favorable number hides a weakness elsewhere.

25. Empirical Results

The supplied evaluation record reports the following production values:

Metric	Exact value
Accuracy	0.9483146067415731
Balanced Accuracy	0.9484675723758693
ROC-AUC	0.9319100760148795
Sensitivity	0.9537037037037037
Specificity	0.9432314410480349
F1-Score	0.9471264367816092
MCC	0.8966632720696869

The near agreement between accuracy and balanced accuracy reflects the relatively balanced class-wise performance represented by the reported sensitivity and specificity. The benchmark is strong within the specified experiment but should remain tied to its exact protocol when communicated in a publication or project presentation.

The benchmark values are presented both as exact decimals and human-readable percentages to avoid ambiguity. The exact record should be retained in the evaluation CSV, while the rounded percentages are easier to communicate in the report and project presentation. When comparing future model versions, the same metric definitions and rounding conventions should be used so that changes can be attributed to the model rather than to reporting differences.

26. Confusion Matrix and Error Analysis

	Predicted ASD	Predicted CON
Actual ASD	True Positive	False Negative
Actual CON	False Positive	True Negative

A false negative is an ASD subject classified as a control, while a false positive is a control classified as ASD. Sensitivity is directly affected by false negatives; specificity is directly affected by false positives.

The supplied report describes the error profile as relatively balanced rather than dominated by one class. The exact counts should always be taken from the generated confusion-matrix artifact rather than reverse-engineered from rounded percentages.

Future reporting should include confidence intervals, repeated-split variability, site-level error distributions, and subgroup analysis. Those additions would make the error analysis substantially more informative than a single aggregate matrix.

A confusion matrix is more than a figure. It identifies the direction of the model errors and helps connect the aggregate metrics to concrete counts. Because the supplied report emphasizes sensitivity and specificity, the matrix should be reviewed alongside those values. In future studies, the same matrix can be stratified by site or demographic subgroup to reveal patterns that disappear when all subjects are pooled together.

27. ROC-AUC and Discrimination Analysis

The reported ROC-AUC is 0.9319100760148795. ROC-AUC evaluates how well model scores rank ASD subjects above controls across thresholds rather than evaluating only one operating point.

This distinction matters because a strong AUC does not automatically guarantee calibrated probabilities. A score can rank observations correctly while still being overconfident or underconfident. The Streamlit dashboard should therefore label probability or confidence outputs as model outputs unless independent calibration has been performed.

The reported AUC complements the sensitivity and specificity values. Together, the metrics describe both threshold-specific performance and the broader discrimination behavior of the classifier.

ROC-AUC is especially useful when the dashboard or a downstream application may consider thresholds other than the default decision boundary. However, ranking quality and calibration are different properties. A confidence bar in the interface should therefore be interpreted as a model-derived score unless calibration has been explicitly demonstrated. This distinction becomes important if the interface is later adapted for decision support.

28. Deep Neuro-Fuzzy Network (DNFN)

The DNFN branch combines neural representation learning with fuzzy membership functions. Its documented hidden representation has $D = 64$ dimensions and its fuzzy stage evaluates $R = 32$ Gaussian rules.

The Gaussian membership for hidden feature d and rule r is:

$$\mu_{d,r} = \exp(-(h_d - c_{d,r})^2 / (\sigma_{d,r}^2 + \epsilon))$$

Here, c is a learnable or optimizable center and σ controls the width of the Gaussian. Membership is larger when the hidden feature lies closer to the rule center.

This branch is useful as a research alternative because it makes gradual rule activation explicit. It is not, however, the source of the 94.8315% production benchmark reported earlier in this report.

The DNFN branch changes the modeling vocabulary from a hard decision boundary to soft rule activation. A hidden feature can partially belong to several Gaussian regions at once, producing a graded representation. This can be useful for exploratory modeling because neuroimaging signals are noisy and heterogeneous. The branch remains experimental, and its performance should not inherit the production SVM benchmark simply because both systems use the same dataset.

29. Gaussian Fuzzy Membership and Rule Aggregation

For every fuzzy rule r , membership values are aggregated across the 64 hidden dimensions using the documented sum:

$$w_r = \sum_{d=1}^{64} \mu_{d,r}$$

The resulting vector of rule activations is a soft representation of the subject. Unlike a hard threshold, a Gaussian membership function permits partial activation, which can be useful when the data do not fall cleanly into a single region of feature space.

The fuzzy representation also introduces a natural parameter-search problem. The location of the rule centers and the widths of the membership functions shape the rule system. The project addresses that problem through the HGSO research branch.

The rule layer creates an intermediate representation that can be inspected more easily than a large collection of hidden weights. Membership values show how close a hidden feature is to a rule center, while aggregated firing strength summarizes the rule activation. The value of this representation is primarily exploratory: it offers a way to inspect how samples occupy the learned feature space and how those regions might relate to ASD versus control classes.

30. Henry Gas Solubility Optimization (HGSO)

Henry Gas Solubility Optimization is inspired by the idea of gas solubility and diffusion in a liquid medium. In the project, fuzzy centers are treated as search variables and are updated through a temperature- and solubility-dependent mechanism.

Temperature schedule: $T = \exp(-\text{epoch} / E_{\text{max}})$

Solubility update: $S_{t+1} = S_t \exp(-1/(T + \epsilon))$

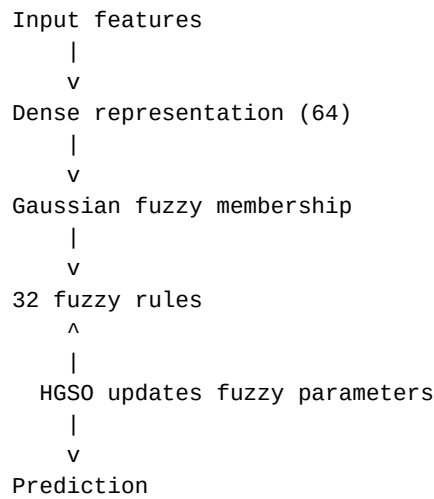
Fuzzy-center update: $c_{d,r}(t+1) = c_{d,r}^t + \epsilon N(0,1) S_{t+1} P_{d,r}$

The documented step coefficient is $\epsilon = 0.01$. The purpose is to provide a stochastic search mechanism that can explore fuzzy parameters rather than relying only on local gradient updates.

HGSO adds a population-style search idea to the fuzzy network. Instead of treating every parameter update as a small gradient step, the fuzzy centers can move through a stochastic process governed by temperature, solubility, and partial-pressure terms. The motivation is to explore non-convex parameter landscapes more broadly. The implementation remains a research branch and therefore requires its own convergence and performance analysis.

31. FHGO-DNFN Integration

The FHGO-DNFN branch places the HGSO search mechanism around the DNFN fuzzy parameters. The conceptual sequence is:



The hybrid is attractive as a research model because the fuzzy-rule centers have an interpretable role while the optimizer provides an additional search mechanism. To make a fair claim about performance, the model would need to be evaluated on matched partitions with repeated runs and the same metric set used for the SVM benchmark.

The hybrid model is best evaluated by keeping the roles of the two components distinct. DNFN defines how the features are represented through fuzzy membership and rule activations, while HGSO changes the search path used to tune those fuzzy parameters. A fair experiment should record not only final accuracy but also training stability, runtime, convergence behavior, and sensitivity to random initialization.

32. Jellyfish Search Optimization (JSO)

Jellyfish Search Optimization is inspired by movement in ocean environments. The project uses a time-control factor to switch between a broad exploration phase and a local exploitation phase.

$$c(t) = |(1 - t/t_{\max})(2 \text{ rand}() - 1)|$$

When $c(t) \geq 0.5$, an ocean-current exploration update is used. When $c(t) < 0.5$, the implementation switches to a local swarm-style update.

The significance of this branch is architectural: it provides a non-gradient search mechanism that can modify an attention representation. That attention concept becomes concrete in the FHJO-CNN branch described next.

The JSO mechanism is useful in this project because the attention mask is itself an optimization variable. The time-control factor creates an explicit transition from broader search toward more local updates. Conceptually, the algorithm therefore provides a way to explore attention layouts before settling into local refinement. This is a different optimization problem from tuning an SVM hyperplane.

33. FHJO-CNN and Spatial Attention

The FHJO-CNN branch combines Jellyfish Search Optimization with a spatial attention mask over a pivotal connectivity representation. The documented attention representation uses a 50 x 50 arrangement for the CNN branch.

For ocean-current exploration, the mask moves toward a mean or global representation using a random factor. For swarm exploitation, the mask receives a stochastic local perturbation. The broad idea is to allow the search process to control where the CNN receives stronger spatial emphasis.

This branch is conceptually different from the production SVM. Instead of flattening the final representation and learning a kernel boundary, it treats the connectivity structure as an image-like object and asks a CNN to learn spatial patterns.

The attention branch treats the selected connectivity structure as an image-like object. The intention is not to claim that a connectivity matrix is literally a photograph, but to take advantage of convolutional inductive bias for local spatial patterns within the arranged representation. The success of this design must therefore be established empirically. The report documents the architecture without assigning it the production benchmark.

34. EfficientNet-B0 Transfer Learning

The FHJO-CNN branch uses EfficientNet-B0 as its convolutional backbone. The attended connectivity representation is resized to 224×224 so that it can pass through the transfer-learning network.

Pivotal connectivity representation

|

v

Spatial attention mask

^

|

JS0

|

v

Attended representation

|

224×224

|

v

EfficientNet-B0

|

v

Learned features

|

v

Classification

Transfer learning is attractive because it provides a strong feature-extraction backbone without requiring the entire convolutional hierarchy to be learned from scratch. At the same time, connectivity matrices are not natural photographs, so the appropriateness of the inherited visual representation must be demonstrated experimentally.

EfficientNet-B0 provides the convolutional feature extractor for the FHJO-CNN branch. Resizing to 224×224 creates the expected spatial input size for the transfer-learning backbone. Because the source representation is a constructed connectivity matrix rather than a natural image, future work should compare pretrained transfer learning against training from scratch or domain-specific encoders to determine whether the visual prior is beneficial.

35. Sparse Autoencoder (SAE)

The Sparse Autoencoder branch learns a compact latent representation by reconstructing the input. The basic structure is encoder -> sparse latent space -> decoder.

The reconstruction term encourages the latent space to retain information that helps reproduce the input. A sparsity penalty encourages only a subset of latent units to be active, which can reduce redundancy in a highly correlated feature space such as functional connectivity.

The SAE is not the final production classifier. It acts as a representation-learning component of the SAE-FRNN architecture, where the latent sequence is subsequently transformed by fuzzy gates and processed recurrently.

The sparse autoencoder introduces a self-supervised reconstruction pressure before classification. This can be helpful when the original feature space contains redundant relationships, which is plausible for pairwise connectivity because many edges share nodes. The L1 penalty encourages a compact latent representation. In the combined FRNN system, that latent representation becomes the input to fuzzy gates and recurrent processing.

36. Fuzzy Recurrent Neural Network and GRU

The SAE-FRNN branch combines sparse representation learning with fuzzy antecedent gates and a two-layer GRU. The documented latent representation is approximately $B \times 50 \times 32$, where B denotes batch size.

The composite objective is: $L_{\text{FRNN}} = L_{\text{CE}}(\hat{y}, y) + 0.1 \|X - \hat{X}\|_F^2 + 0.01 \|Z_{\text{latent}}\|_1$

The three terms correspond to classification, reconstruction, and latent sparsity. The GRU then models dependencies across the ordered latent positions. This gives the branch a sequential inductive bias that is not present in the static SVM feature vector.

As with the DNFN and FHJO branches, the FRNN architecture is a research component. Its performance must be established through an independent experiment rather than inferred from the production SVM numbers.

The FRNN branch adds temporal or ordered dependence at the latent level. The GRU does not receive raw voxel data; it receives a structured latent sequence. This separation lets the model focus its recurrent capacity on a compact representation rather than the full high-dimensional connectivity space. The composite loss also gives the latent representation two jobs: retain information useful for classification and remain reconstructive and sparse.

37. Comparative Model Architecture

Model	Core mechanism	Role
RBF-SVM	Nonlinear kernel classification	Production benchmark
DNFN	Neural layers + Gaussian fuzzy rules	Research
HGSO	Gas-solubility-inspired search	Fuzzy parameter optimization
FHGO-DNFN	DNFN + HGSO	Hybrid research model
FHJO	Jellyfish-inspired optimization	Attention optimization
FHJO-CNN	FHJO + attention + CNN	Research
EfficientNet-B0	Transfer-learning CNN	CNN backbone
SAE	Encoder-decoder + L1 sparsity	Latent representation
SAE-FRNN	SAE + fuzzy gates + GRU	Sequential research model

The project is best understood as two connected branches: a stable production path centered on the RBF-SVM, and an experimental suite that tests richer modeling ideas. The distinction keeps the benchmark interpretable and makes future comparisons fairer.

The model comparison highlights a practical research philosophy. More complex architectures are not automatically better; they provide different inductive biases and may require more data, compute, and tuning. The RBF-SVM offers a strong, compact benchmark, while DNFN, HGSO, FHJO-CNN, and SAE-FRNN open alternative research directions. The value of the repository comes from being able to compare these choices under the same data and evaluation discipline.

38. Software Architecture and Module Responsibilities

Module	Responsibility
dataset_loader.py	Dataset discovery, subject matching, functional-file loading
preprocessing.py	Percentile clipping and signal preprocessing
roi_extraction.py	CC200 ROI handling and standardization
functional_connectivity.py	Pearson correlation, Fisher transformation, edge extraction
pivotal_region.py	Welch t-tests and top-600 selection
feature_extraction.py	Selected-edge preparation and scaling
dnf_network.py	Deep Neuro-Fuzzy Network
fhgo.py	Henry Gas Solubility Optimization
fhgo_dfn.py	HGSO + DNFN integration
fhjo.py	Jellyfish Search Optimization
cnn_transfer_learning.py	EfficientNet-B0 integration
frnn.py	Sparse AE + fuzzy recurrent architecture
evaluation.py	Metrics and confusion-matrix evaluation
frontend.py / main.py	Dashboard / orchestration entry points

The modular structure is important for the input-format extension. File-type detection and adapter code can be placed at the application boundary rather than changing the scientific core of the classifier.

Separating modules by responsibility makes debugging easier. A correlation error can be isolated in `functional_connectivity.py` without inspecting the SVM code; a dashboard upload problem can be isolated in `frontend.py` or its adapter without changing feature extraction. This is particularly valuable for the current update because the file-format contract changes at the boundary of the system rather than in the learned model itself.

39. Project Directory and Artifact Management

```

ABIDE_ASD_Detection/
├── data/
│   ├── raw/
│   ├── processed/
│   └── features/
├── src/
│   ├── dataset_loader.py
│   ├── preprocessing.py
│   ├── roi_extraction.py
│   ├── functional_connectivity.py
│   ├── pivotal_region.py
│   ├── feature_extraction.py
│   ├── dnf_network.py
│   ├── fhgo.py
│   ├── fhgo_dfn.py
│   ├── fhjo.py
│   ├── cnn_transfer_learning.py
│   ├── frnn.py
│   └── evaluation.py
├── results/
│   ├── figures/
│   ├── tables/
│   └── models/
├── frontend.py
├── main.py
├── requirements.txt
├── README.md
└── LICENSE

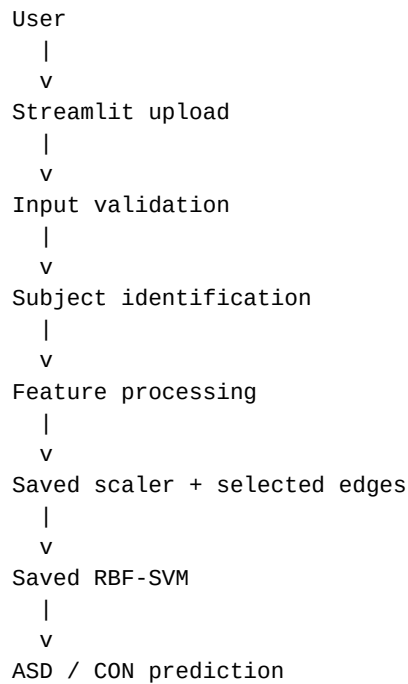
```

The main serialized artifact is expected at `results/models/abide_svm_pipeline.pkl`. Evaluation metrics can be stored at `results/tables/evaluation_metrics.csv`, and the confusion-matrix visualization under `results/figures/`. Raw subject data should remain outside version control.

Artifact management is part of reproducibility. The model file alone is insufficient if the selected edge indices or scaler are missing. Likewise, a metric CSV without a code version or split definition is hard to interpret. Keeping models, tables, figures, and intermediate features in separate directories makes the repository easier to inspect while keeping restricted subject data outside version control.

40. Streamlit Dashboard Architecture Before the Input Extension

The dashboard is the user-facing layer around the saved production pipeline. Before the extension, the documented workflow was centered on functional-data input, subject identification, feature processing, the saved scaler, the saved SVM, and a class prediction.



The dashboard is deliberately not treated as a replacement for scientific preprocessing. It must respect the model input contract and produce a compatible feature representation before prediction. The updated version keeps this principle while broadening the accepted upload suffixes.

The dashboard should be viewed as a thin client over a scientific pipeline. Its job is to guide the user through file selection, validation, transformation, and prediction, while the learned artifacts remain in a controlled backend layer. This separation reduces the chance that a UI change will unintentionally modify the production classifier. It also creates a clean place to introduce the new .nii, .jpg, and .png input contract.

41. Updated Input Contract: .nii, .jpg, and .png Only

The revised application requirement is explicit: the dashboard accepts exactly three file extensions: .nii, .jpg, and .png. No other input format is part of the supported interface contract.

Extension	Accepted	High-level role
.nii	Yes	Neuroimaging volume input
.jpg	Yes	Image input through the application adapter
.png	Yes	Image input through the application adapter
All other formats	No	Reject before inference
Design rule File extension support is intentionally narrow. The application should perform an early allow-list check, followed by content validation, before any data reaches the classifier.		

The updated contract does not authorize changes to the model architecture or benchmark configuration. It is a user-interface and input-adaptation requirement.

The allow-list is intentionally strict. In practice, this means the application should inspect the extension before attempting to decode the content and reject everything outside the three permitted types. A strict contract simplifies testing and prevents ambiguous behavior for text files, archives, spreadsheets, or arbitrary binary data. The extension check should be followed by actual content validation because a file name alone does not prove that the payload is valid.

42. Input Adapter Layer and Representation Integrity

Accepting three file suffixes does not mean that the three files contain the same kind of data. The input-adapter layer is therefore responsible for converting each accepted input into a representation that can be processed by the downstream inference path without changing the protected model.

Stage	.nii path	Image path (.jpg / .png)
1. Extension validation	Allow	Allow
2. Content validation	Check valid NIfTI	Check valid image data
3. Representation adapter	Map to required inference representation	Map through configured image adapter / representation path

The report does not invent a new classifier for JPG or PNG input. Instead, it treats the adapter as an interface component. The scientific feature space remains protected: if the downstream production SVM expects 600 standardized connectivity features, the adapter must eventually deliver that compatible representation or route the input through a separately defined model path that does not alter the reported SVM benchmark.

This separation is essential to the requirement that the updated application introduce no unintended changes to the earlier accuracy.

Representation integrity is the central technical challenge of the input extension. A valid NIfTI volume can be routed through the neuroimaging preprocessing path, while an image file requires an image interpretation path. The adapter must not invent a misleading mapping just to satisfy a shape requirement. Instead, it should produce only representations that the configured downstream model is designed to consume, with compatibility checks before prediction.

43. Inference Outputs and User-Facing Behavior

The primary output of the system is a binary prediction: ASD or Typical Control. Where the configured classifier exposes probability estimates, the dashboard can also display model probabilities or a confidence visualization.

Output	Description
Predicted class	ASD or CON
Model score	Decision value / score where available
Probability	Displayed only when supported by the model
Input status	Accepted, invalid, or incompatible
Processing status	Validation, transformation, prediction

The dashboard should clearly separate technical confidence from clinical certainty. A model probability is not automatically a clinically calibrated risk measure. For the same reason, the user interface should not label the output as a medical diagnosis.

Expected user experience

A valid supported file should move through validation, preprocessing/adaptor logic, inference, and result display without requiring the user to understand the internal 19,900-to-600 feature transformation.

The dashboard output is intentionally simple: a predicted class, supporting score or probability where available, and clear processing status. Simplicity is useful because the underlying pipeline is mathematically complex. At the same time, the interface should retain enough metadata for researchers to understand what input was processed and whether the file reached the expected inference path. This is especially important when three different container formats are allowed.

44. Accuracy Preservation and Regression Strategy

The updated input requirement explicitly prioritizes preservation of the previously reported benchmark. The safest engineering strategy is to freeze the production model and its learned preprocessing objects, then place new file support around them.

Regression target	Required behavior	Reason
Model artifact	Unchanged	Protect learned decision function
Selected edge indices	Unchanged	Preserve feature mapping
StandardScaler	Unchanged	Preserve feature distribution
SVM hyperparameters	Unchanged	Preserve benchmark configuration
Evaluation outputs	Reproducible	Detect unintended accuracy drift

The acceptance of a .nii, .jpg, or .png upload should therefore be tested as an interface change first. Existing benchmark inputs should be rerun after any modification. If the result changes, the difference must be explained rather than hidden.

The preservation target is the supplied benchmark: 94.8315% accuracy, 94.8468% balanced accuracy, 0.9319 ROC-AUC, 95.3704% sensitivity, 94.3231% specificity, 0.9471 F1, and 0.8967 MCC.

Regression testing should compare the updated application against a known-good baseline rather than only checking that the new UI loads. The strongest test is to rerun the existing benchmark input and confirm that the output remains consistent. Additional smoke tests should exercise one valid file for each new format, invalid payloads, unsupported extensions, missing model artifacts, and malformed feature arrays.

45. Error Handling and No-Error Requirement

A deployment change is incomplete if the application accepts files but crashes under malformed, empty, oversized, incompatible, or unsupported inputs. The updated application therefore needs explicit validation at multiple layers.

- Extension check: reject every suffix other than .nii, .jpg, and .png.
- File-content check: verify that the payload is readable and structurally valid for its declared type.
- Shape check: confirm that the transformed representation satisfies the expected model input dimensions.
- Numeric check: reject NaN, infinite, or otherwise invalid feature values before prediction.
- Artifact check: confirm that the saved model and preprocessing objects are available before inference.
- Exception handling: convert predictable failures into clear user-facing messages rather than uncaught tracebacks.
- Regression check: verify that existing benchmark execution still completes successfully after interface changes.

Acceptance criterion

The application should fail gracefully on bad input and run cleanly on valid input. A new file-format branch must not introduce import errors, tensor-shape errors, missing-model errors, or silent fallback behavior that changes the production prediction logic.

The no-error requirement is best handled as a layered contract. Each layer should reject only the failures it can understand and report the problem in terms useful to the next layer. For example, the uploader checks the extension, the decoder checks file integrity, the adapter checks representation, and the model wrapper checks feature dimensions. This prevents low-level exceptions from leaking directly into the user interface.

46. Installation, Execution, and Deployment

The environment is based on Python and the scientific computing stack used by the project. A virtual environment keeps versions isolated and makes deployment easier to reproduce.

```
python -m venv venv
# Windows
venv\Scripts\activate
# Linux/macOS
source venv/bin/activate
pip install -r requirements.txt

# Train / reproduce the production pipeline
python main.py

# Launch the dashboard
python -m streamlit run frontend.py
```

Core dependencies include NumPy, Pandas, SciPy, scikit-learn, Matplotlib, Seaborn, Joblib, PyTorch, torchvision, and Streamlit. The SVM path is comparatively CPU-friendly; the deep-learning research branches may benefit from an NVIDIA GPU with a compatible PyTorch installation.

For the updated dashboard, deployment testing should include one valid file from each allowed type, one malformed file for each type, and at least one intentionally unsupported extension to confirm correct rejection.

Deployment quality depends on making the successful path repeatable. The environment should install from requirements.txt, the training script should produce documented artifacts, and the dashboard should load those artifacts without retraining. After the input extension, the deployment smoke test becomes broader but the core execution commands remain the same. The goal is to increase interface capability without increasing uncertainty about the model.

47. Ethics, Privacy, and Clinical Safety

Neuroimaging data is sensitive research data. Even when a dataset is publicly available and de-identified, users should follow the governing data-use terms and should not attempt to identify participants. Raw subject files should not be committed to a public repository.

The dashboard must present its predictions as outputs of a research model. The software should not claim that an individual has ASD or does not have ASD based solely on the classifier output. Clinical interpretation requires qualified professionals and substantially stronger validation.

The updated .nii, .jpg, and .png input contract makes input validation a privacy and safety issue as well as a software issue. A file should not be displayed or processed beyond what is needed for the intended inference path, and logs should avoid storing sensitive information unnecessarily.

If the project is ever advanced toward clinical use, external validation, prospective evaluation, calibration, subgroup analysis, security testing, clinical workflow analysis, and appropriate regulatory review would be required.

Ethics also applies to engineering detail. A public repository should not accidentally expose subject identifiers, raw scans, logs containing sensitive paths, or credentials. The dashboard should minimize the information it retains about uploads. Most importantly, the language shown to users should avoid turning a classification score into a statement of medical fact. Technical accuracy and responsible communication are both required.

48. Limitations and Future Research

The first limitation is multisite heterogeneity. A classifier may learn site-specific differences that do not represent ASD-related biology. The second is evaluation scope: one train/test split is informative but weaker than repeated cross-validation or external validation. The third is leakage risk when global feature selection is performed before partitioning.

The fourth limitation is the input-format extension itself. Supporting .nii, .jpg, and .png at the interface is a software capability; it does not automatically make all three formats biologically equivalent. The representation adapter must be validated separately for every supported input class.

Future work should prioritize leakage-safe nested validation, site-held-out evaluation, ABIDE-I to ABIDE-II external testing, calibration, subgroup analysis, stable ROI mapping, and independent replication. Once those foundations are strong, additional work can investigate graph neural networks, dynamic connectivity, multimodal MRI, uncertainty estimation, and richer interpretability.

The research-model suite also deserves matched benchmarking. DNFN, FHGO-DNFN, FHJO-CNN, and SAE-FRNN should be compared under common splits and reporting standards so that architectural complexity can be judged on evidence rather than on novelty alone.

The future-work plan deliberately starts with validation rather than with additional architectural complexity. A model that survives cross-site testing, leakage-safe nested validation, calibration analysis, and subgroup checks provides a stronger research contribution than a more elaborate network evaluated on a single split. Once the baseline is trustworthy, the advanced model branches can be compared fairly and the input adapters can be validated against clear scientific targets.

49. Conclusion and References

The ABIDE ASD Detection Pipeline demonstrates a complete progression from resting-state fMRI to functional connectivity, statistical edge selection, nonlinear classification, evaluation, and interactive deployment. The core production path is intentionally compact: CC200 regional time-series, percentile clipping, ROI standardization, Pearson correlation, Fisher transformation, 19,900-edge construction, top-600 t-test selection, StandardScaler, and RBF-SVM.

The reported benchmark is 94.8315% accuracy, 94.8468% balanced accuracy, 0.9319 ROC-AUC, 95.3704% sensitivity, 94.3231% specificity, 0.9471 F1-score, and 0.8967 MCC. The project also provides research implementations of DNFN, HGSO, FHGO-DNFN, FHJO-CNN, EfficientNet-B0, SAE, and SAE-FRNN.

The updated deployment requirement is incorporated as an interface-layer change: exactly .nii, .jpg, and .png are accepted, while other file types are rejected. The classifier, selected-edge mapping, scaling, and benchmark are treated as protected components, and all new file handling is subject to validation and regression testing.

References and technical resources:

- Autism Brain Imaging Data Exchange (ABIDE) - primary neuroimaging research resource referenced by the project.
- Craddock, R. C. and collaborators - functional parcellation methods underlying the CC200 representation.
- Cortes, C. and Vapnik, V. (1995). Support-vector networks. Machine Learning.
- PyTorch and torchvision - deep-learning implementation framework used by the research architectures.
- scikit-learn - machine-learning and preprocessing library used for the production pipeline and evaluation.
- SciPy - scientific and statistical computing library supporting hypothesis testing.
- NumPy and Pandas - numerical and tabular data-processing foundations.
- Matplotlib and Seaborn - evaluation and visualization tools.
- Joblib - model and preprocessing artifact serialization.
- Streamlit - interactive application and deployment framework.

Final note: this report is a technical description of the supplied project and its updated deployment requirement. It should not be interpreted as evidence of standalone clinical diagnostic validity.